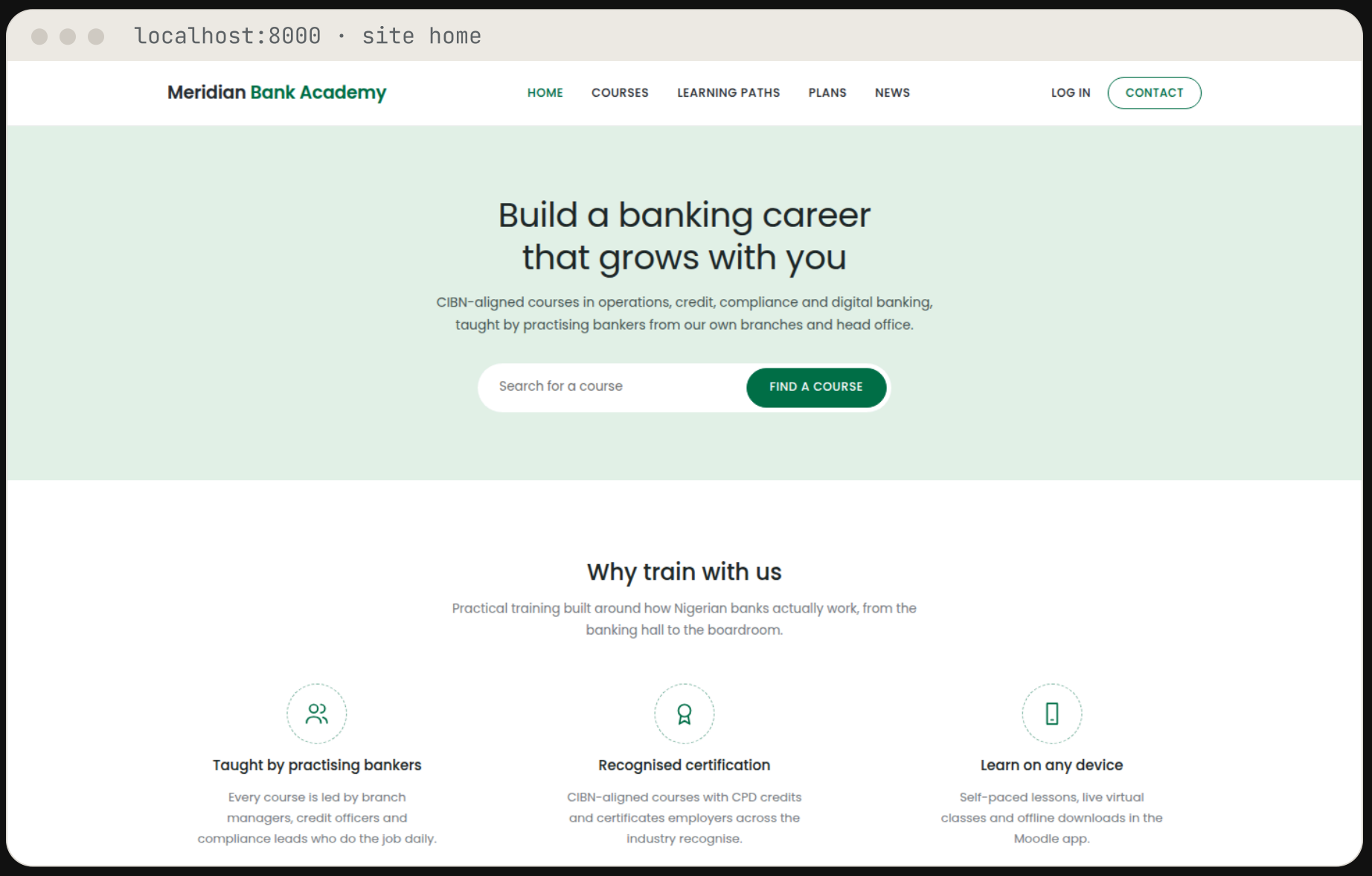
MOODLE 5.3 · THEME ARCHITECTURE

How Retani Studio is built on Moodle.

Renderers, Mustache templates, React islands and the two JavaScript systems, explained with the theme's own code and pages.

PHP renderers · Mustache · React in Mustache · AMD vs ESM · Template overrides



Retani Studio · a Moodle 5.3 theme

Swipe →

Every page goes through the same five steps

A theme doesn't rewrite Moodle. It steps in at the points Moodle leaves open.

1

Page script

Moodle core loads the data, e.g. the course list.

`course/index.php`

2

Layout

The theme picks the page frame: header, sidebars, footer.

`layout/coursecategory.php`

3

Renderer

PHP turns data into something a template can show.

`classes/output/`

4

Mustache template

Plain HTML with placeholders, overridable per theme.

`templates/*.mustache`

5

JavaScript

Adds interactivity: AMD modules or React (ESM).

`amd/ · js/esm/`

One folder per job

Since Moodle 5.1, all web code sits under **public/**. The theme is a plugin in **public/theme/retani_studio**.

```
retani_studio/
├── config.php            # page type → layout file
├── layout/               # page frames (frontpage, login, drawers…)
├── classes/output/       # renderers + data for templates
│   └── core/course_renderer.php
├── templates/            # Mustache; core/… overrides core
│   ├── core/loginform.mustache
│   └── catalogue.mustache
├── amd/src/              # classic JS modules  → amd/build
├── js/esm/src/           # React + TypeScript  → js/esm/build
├── scss/                 # styles, compiled by Moodle
├── db/                   # install, upgrade, hooks, services
└── lang/en/              # every visible string
```

✓ Built files (**amd/build**, **js/esm/build**) are committed, so a site needs no Node at runtime.

✓ Text lives in **lang/**, so the whole theme can be translated.

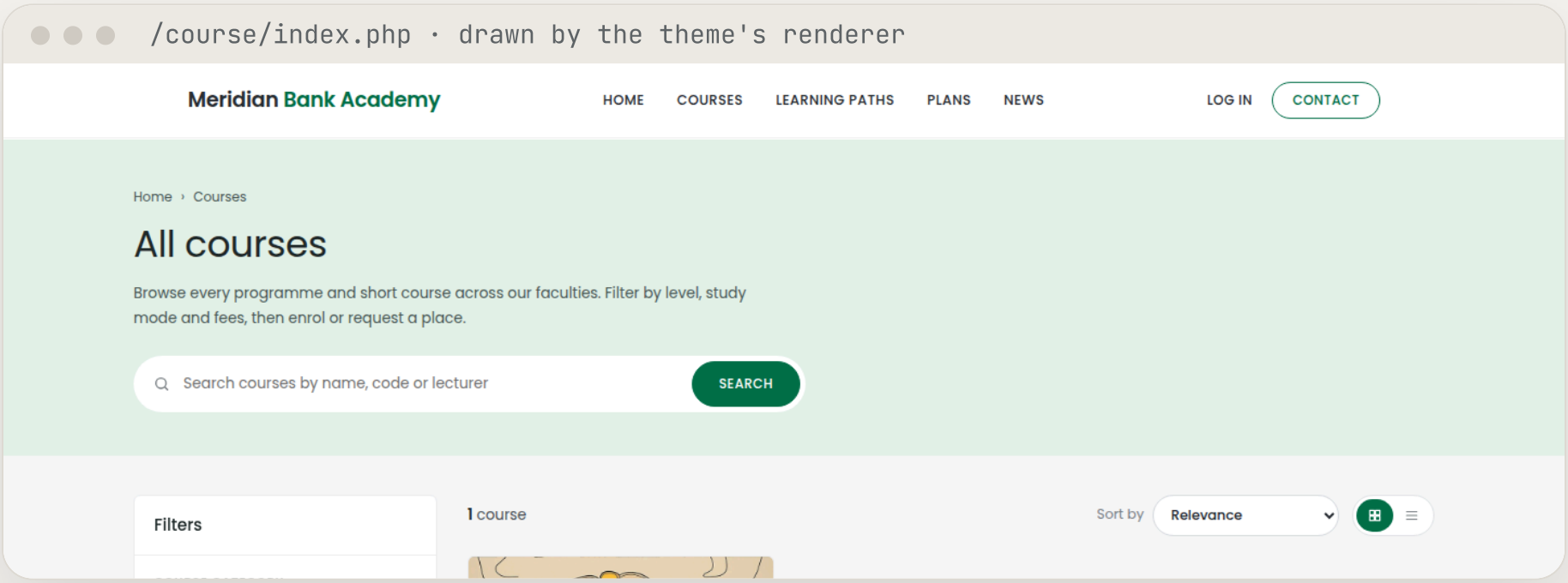
Renderers decide what is drawn

A renderer is a PHP class Moodle asks "draw this". The theme can swap in its own by using the same name in its own namespace.

classes/output/core/course_renderer.php

```php
class course_renderer extends \core_course_renderer {
    public function course_category($category) {
        // Not our page? Let Moodle draw it as usual.
        if (!catalogue::is_active($this->page)) {
            return parent::course_category($category);
        }
        // Otherwise draw the Retani course catalogue.
        $catalogue = new catalogue($this->page);
        return $this->render_from_template(
            'theme_retani_studio/catalogue',
            $catalogue->export_for_template($this));
    }
}
```

/course/index.php · drawn by the theme's renderer



04 · OVERRIDING TEMPLATES

Same name, new look

Moodle looks for a template in the active theme first. Copy a core template into **templates/<component>/** and the theme's version wins. No core files are touched.

MOODLE CORE

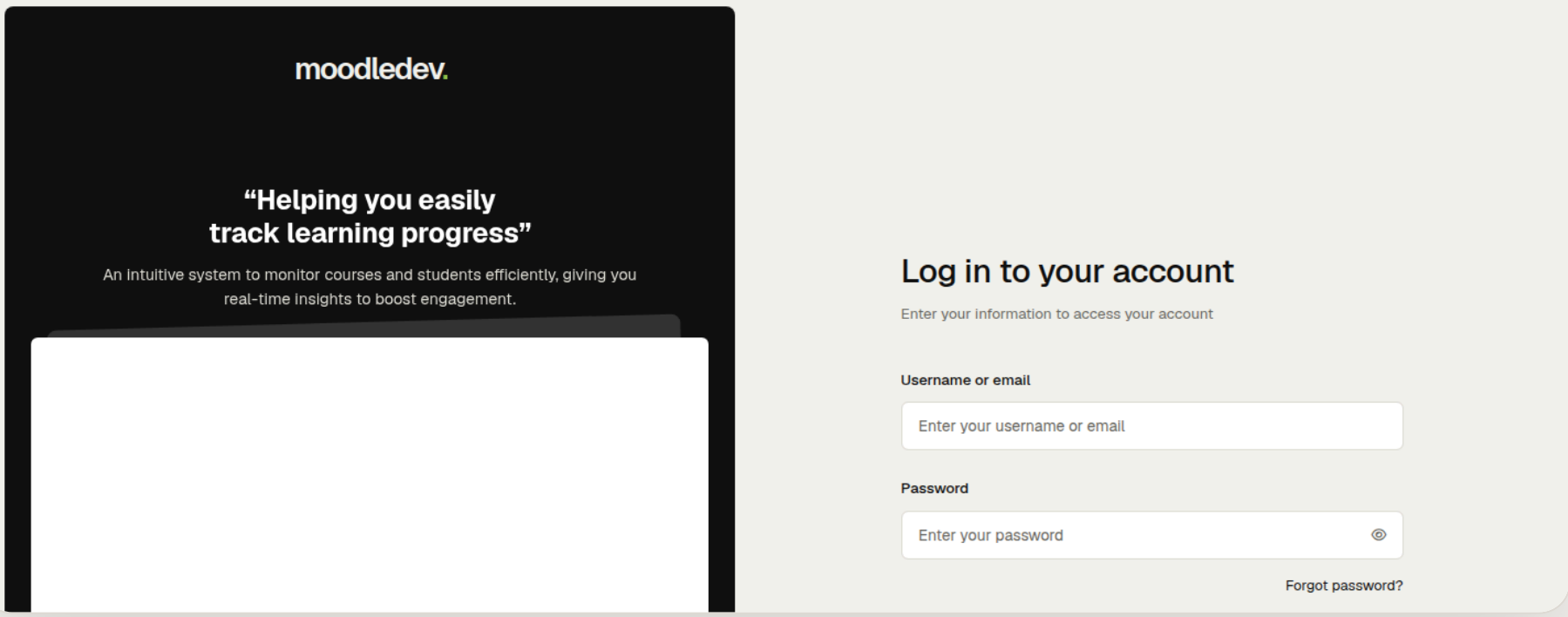
`public/lib/templates/loginform.mustache`

→

RETANI STUDIO (USED)

`theme/retani_studio/templates/core/loginform.mustache`

/login/index.php · core/loginform overridden



core/loginform

core/full_header

block_myoverview/main

core_courseformat/local/content

Keep the original variables, then purge caches.

Drop a React component straight into a template

Moodle 5.x adds a **{{#react}}** helper. You name the component and pass props as JSON; Moodle mounts it when the page loads.

templates/core/login_panel.mustache

```
{{#react}}
{
    "component": "@moodle/lms/theme_retani_studio/LoginCarousel",
    "class": "login-carousel-mount",
    "props": { "interval": 6000, "slides": [ … ] }
}
<!-- Anything here is shown until React takes over,
     and stays if JavaScript is off. -->
{{/react}}
```

`LoginCarousel`

Login page

- Slides, arrows and dots
- Props come from lang strings

`LeftBar`

Dashboard sidebar

- PHP checks permissions first
- React only draws what's allowed

The PHP side still owns the data and permissions. React handles the interaction.

06 · AMD VS ESM

Two JavaScript systems

	AMD (CLASSIC)	ESM (NEWER)
Folder	amd/src → amd/build	js/esm/src → js/esm/build
Language	Modern JavaScript	TypeScript + React
Loaded by	RequireJS, from a template's {{#js}} block	Native imports, or the {{#react}} helper
Best for	Small touches on server-drawn HTML	Rich, stateful widgets
In Retani	catalogue filters, landing menu	LeftBar, LoginCarousel, course overview

amd/src/catalogue.js

```
export const init = (rootId) => {
  const root = document
    .getElementById(rootId);
  initForm(root);
  initFiltersPanel(root);
  initViews(root);
};
```

js/esm/src/leftbar/LeftBar.tsx

```
import {useState} from 'react';
import {fetchOne}
  from '@moodle/lms/core/ajax';

export default function LeftBar(
  props: LeftBarProps) { … }
```

Rule of thumb: if the HTML is already right and needs a nudge, use AMD. If the UI has its own state, use React.

Build with Moodle, not against it

1. **Layouts** set the page frame; fall back to the default when a feature is off.
2. **Renderers** prepare the data; extend core's and call *parent* when it isn't your page.
3. **Templates** are overridden by name, never by editing core.
4. **React** lives inside Mustache via {{#react}}, with plain HTML as a fallback.
5. **AMD** for light touches, **ESM** for rich components.
6. Filters live in the URL, so every view can be shared and works on any screen.

